



MMold - 1.2.6

Microservice Archetype Suite

Initiated, aggregated & tailor-made by Tim Ho & Associates, Apr 2023

A solid dark blue horizontal bar is located at the bottom left of the slide, below the text.



Agenda

- Context
- Why this archetype
- Domain Model (from business perspective)
- Domain Model (from technical perspective) and Mappings of business domain model and technical artifacts
- Hexagonal Architecture and Code Structure based on it
- Package Organization
- Sample Business Domain
- Refs



Context

- For MSA-based applications, there arises the need for a unified and standardized archetype for a microservice code structure for reusability and productivity. The archetype needs be in compliance with DDD approach as well as hexagonal architecture

2 Variants

2 Variants: DDD-based microservice archetype and CRUD-based microservice archetype

DDD-based microservice archetype



MMold 's Bedrocks

- Hexagonal Architecture (Ports and Adapters pattern) by Alistair Cockburn
- Domain-Driven Design Approach by Vernon
- Clean Architecture and Clean Code by Robert C. Martin

Why This Archetype

- Why these variants:
- Materialize DDD artifacts or Domain Model (As domain decomposition approach used is DDD)
- In compliance with Hexagonal Architecture
- A bespoke and well-architected archetype for reusability, development productivity and quality



Domain Model – From Business Perspective

- Business Entities
- Business Rules
- Business Flows
- Business Operations
- Business Events

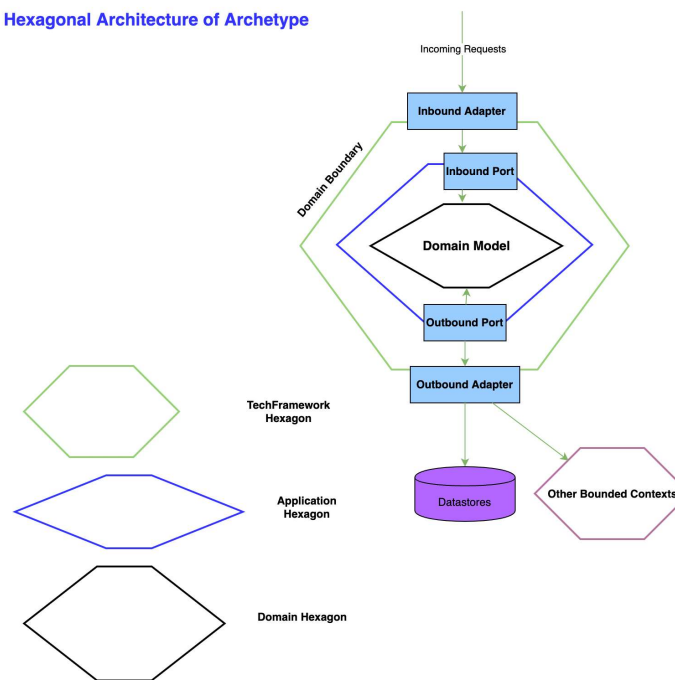


Mappings of Domain Model – From Technical Perspective

- Root Aggregates/Entities/Value Objects (VOs)
- Domain Rules
- Sagas
- Commands/Queries
- Domain Events

Layering /w Hexagonal Architecture

Hexagonal Architecture of Archetype



Hexagonal Architecture-based Code Structure

A microservice archetype consists of 2 main groups of components: Domain Model and Domain Model Services

□ Domain Model (core elements of domain)

Houses core business domain logics, components including:

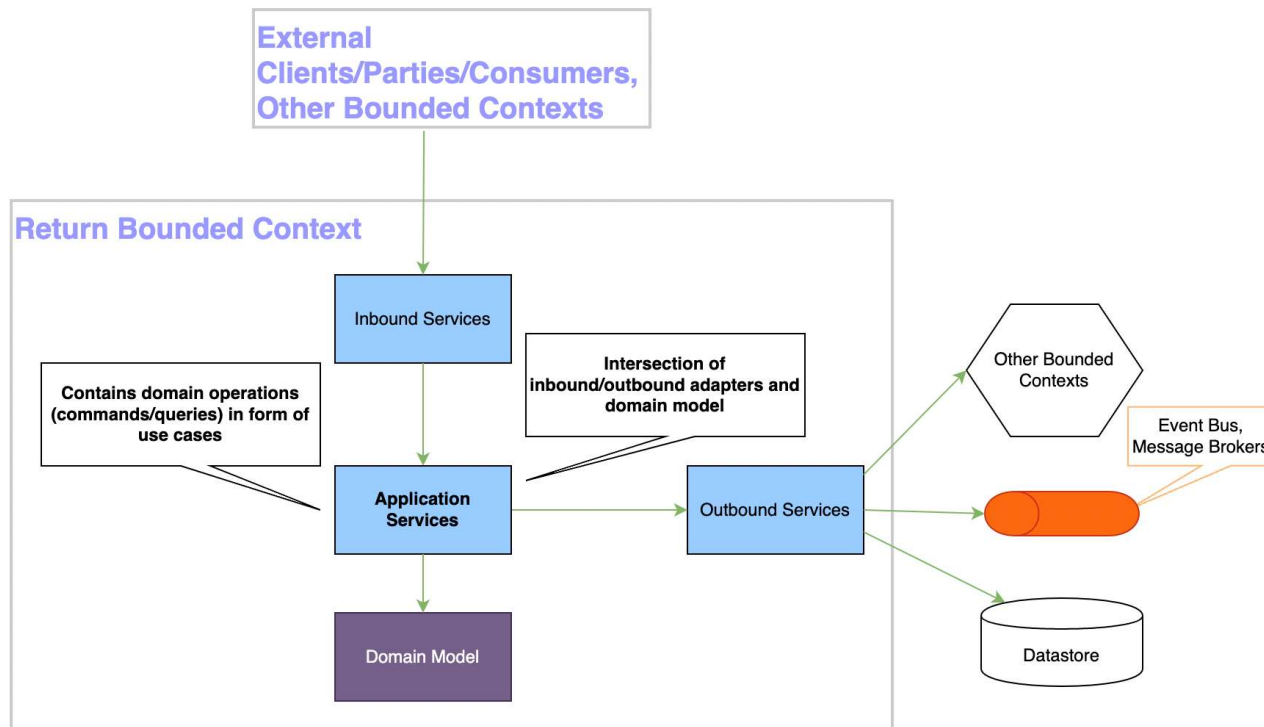
- AggregateRoots/Entities/Value Objects (VOs)
- Domain Rules
- Sagas
- Commands/Queries
- Domain Events

□ Domain Model Services (supporting elements of domain)

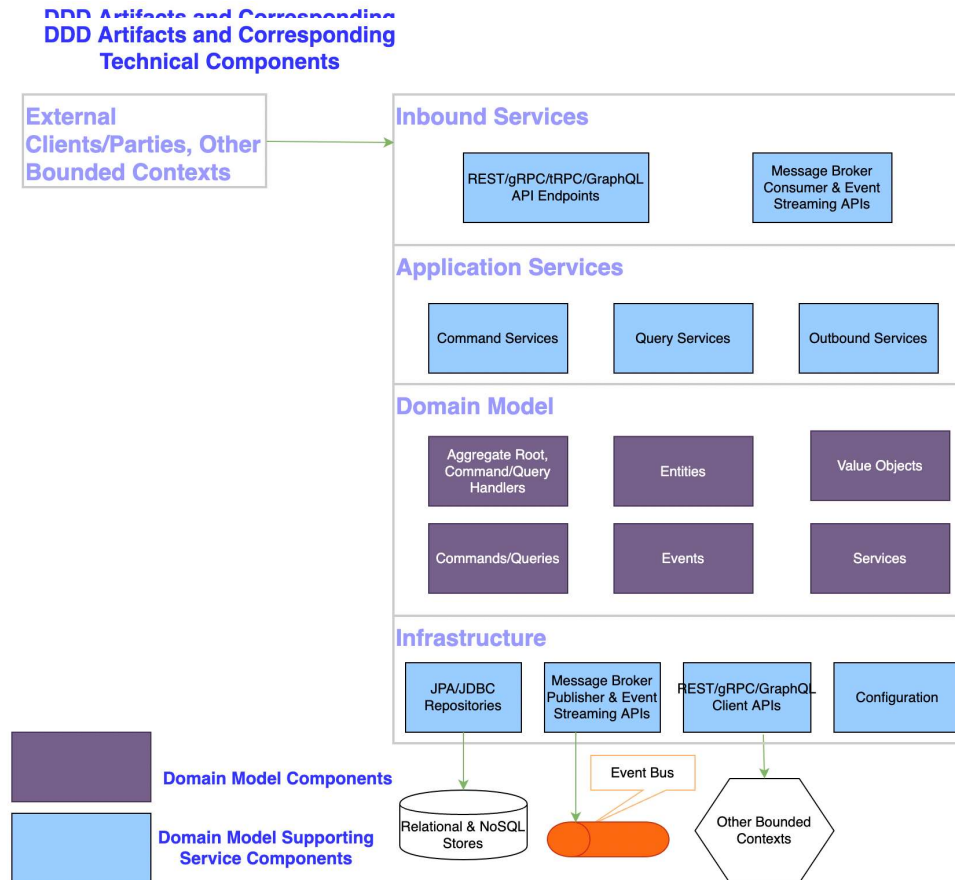
- Inbound Services: expose domain states for external parties to access
- Outbound Services: send domain states external parties
- Application Services: glue that wire Inbound/Outbound Services and Domain Model together

Hexagonal Architecture-based Code Structure (Cont.)

High-Level Code Architecture of Archetype

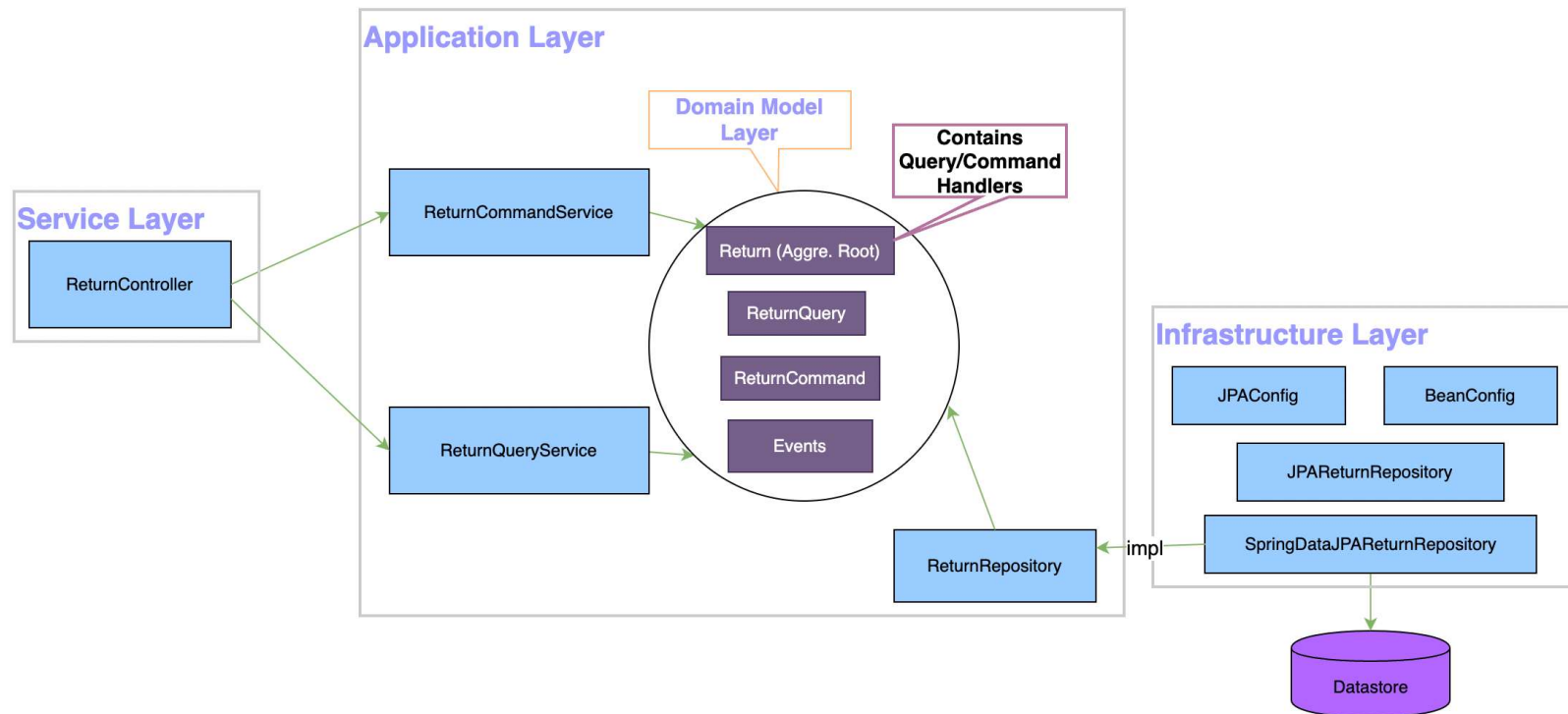


DDD Artifacts and Components



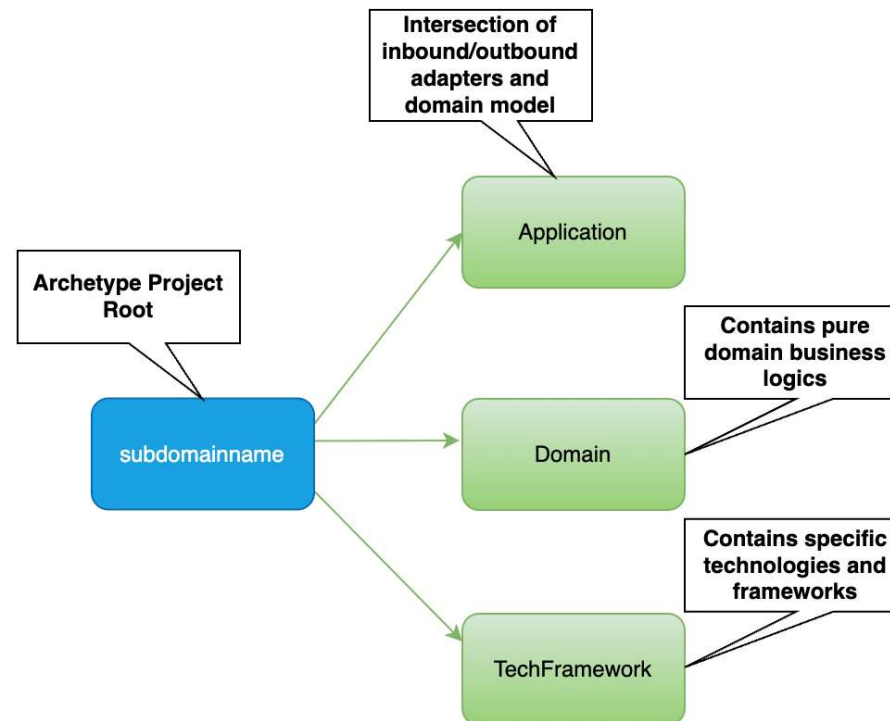
Multi-layered Architecture

Multi-layered Architecture of Archetype



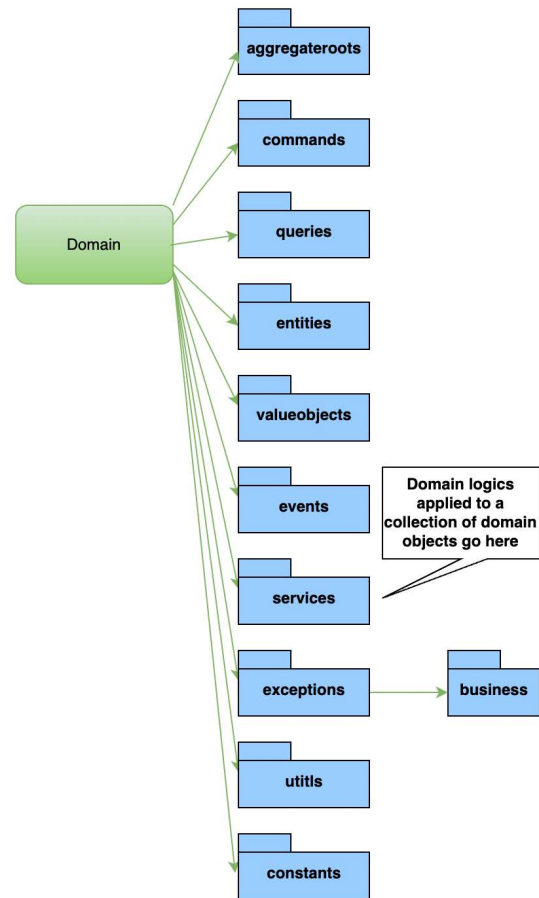
Multi-Module Organization

Multi-module Structure of Archetype



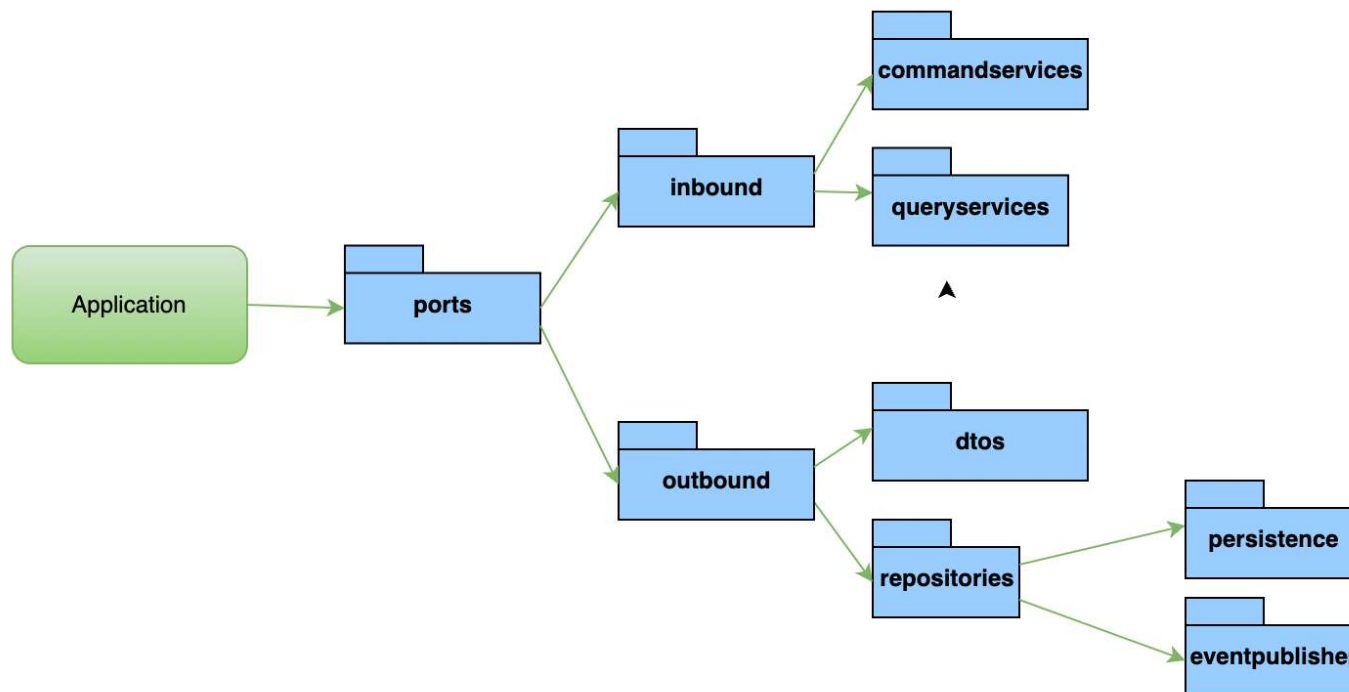
Package Organization – Domain Module

Package Hierarchy of Domain Module



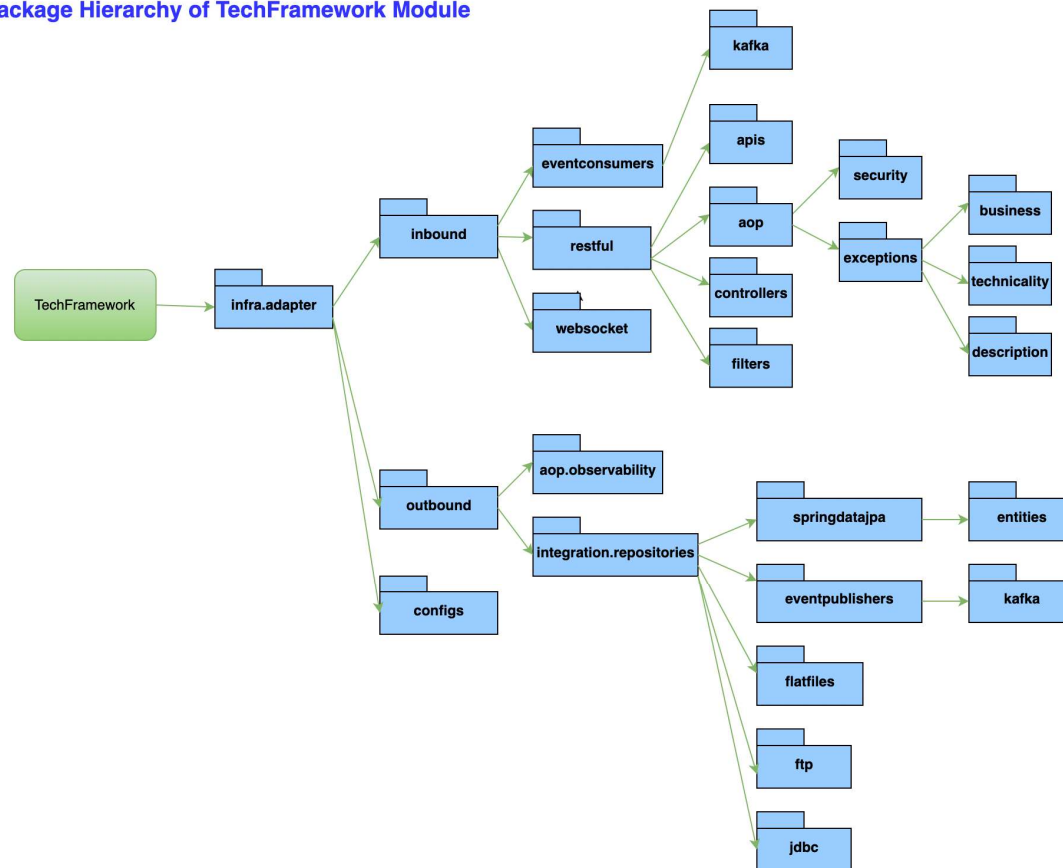
Package Organization – Application Module

Package Hierarchy of Application Module



Package Organization – TechFramework Module

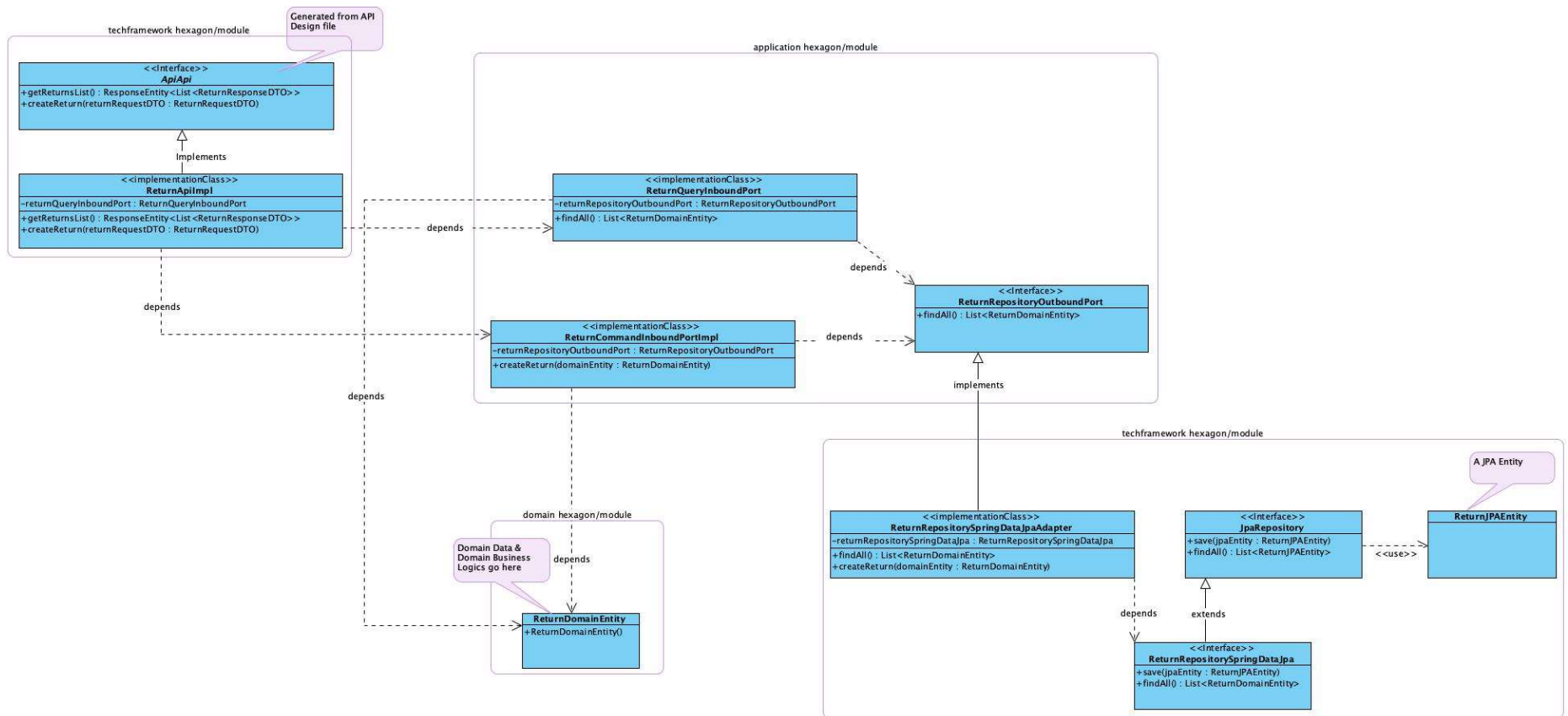
Package Hierarchy of TechFramework Module



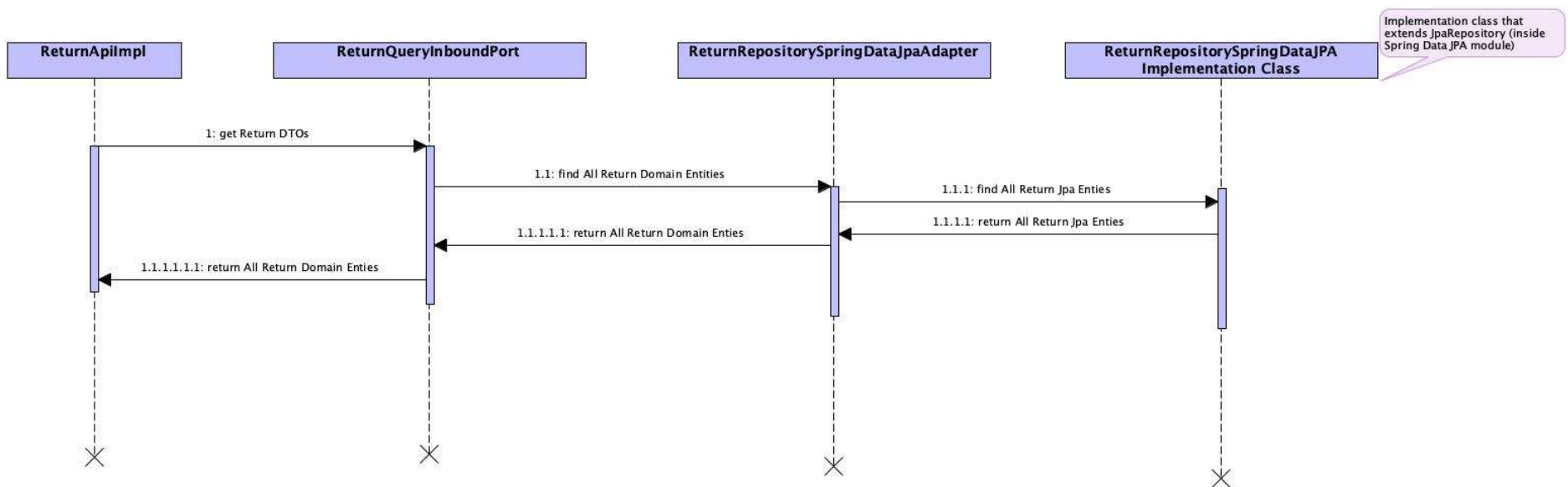
Sample Business Domain – Return Subdomain

- 'Return' in Logistics with its business domain model is used as a Sample Domain for illustrative purpose.

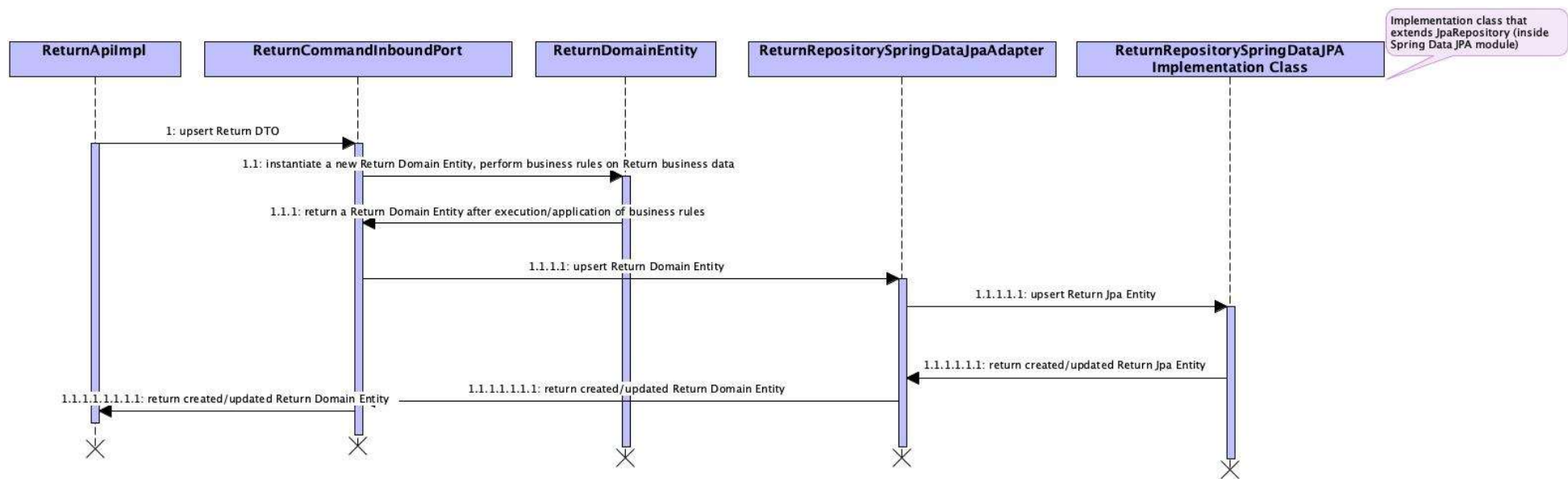
Class Diagram



Sequence Diagram – Query Use Case



Sequence Diagram – Command Use Case





Variant 2



CRUD-based microservice archetype

Codebase Structure

```

└─ companyname-productname-domainname-ms-ddd-v1.2.6
   └─ .gradle
   └─ .idea
   └─ application
      └─ .gradle
      └─ build
      └─ gradle
      └─ src
      └─ wrapper
         └─ build.gradle
         └─ gradlew
         └─ gradlew.bat
         └─ mvnw
         └─ pom.xml
         └─ settings.gradle
   └─ docs
   └─ domain
      └─ .gradle
      └─ build
      └─ gradle
      └─ src
         └─ build.gradle
         └─ gradlew
         └─ gradlew.bat
         └─ mvnw
         └─ pom.xml
         └─ settings.gradle
   └─ gradle
   └─ techframework
      └─ .gradle
      └─ build
      └─ gradle
      └─ src
         └─ wrapper
            └─ build.gradle
            └─ gradlew
            └─ gradlew.bat
            └─ mvnw
            └─ petstore-v3.0.yaml
            └─ pom.xml
            └─ settings.gradle
         └─ .gitignore
         └─ .gitlab-ci.yml
         └─ Dockerfile
         └─ gradlew
```

V1.2.3 – Release Notes

- Create 2 variants (ddd-microservice-archetype and crud-microservice-archetype) to cover real-life contexts
- Replace a database entity 's autoincremental id with UUID for database vendor portability (database vendor decoupling)
- Implement full-flow command/query user stories (use cases)
- Integrate with OpenAPIs for API First and Documentations
- Refine Gradle scripts and Java Modules logics

V1.2.4 – Release Notes

- Integrate with Flyway for database schema migrations
- Auto generate server stub code and client skeleton code from Swagger-based API Design File with Swagger Code Gen Gradle Plugin, include Swagger UI dependencies for API Documentation Display on Web page, refactor REST controller to implement generated API interface and use generated RequestDTO/ResponseDTO
- Integrate with GraphQL for API First and Customized Queries
- Temporarily leave out Java Platform Modules System (Java 9 Jigsaw)
- Added UT boilerplate codes to REST APIs and Data Access Layer
- Convert Kotlin-based Gradle build scripts into Groovy-based ones for simplicity and stability



V1.2.6 – Release Notes

- Support Spring Boot 3.1.0, Spring 6.x
- Integrate with OpenAPI 3 for API First and Code Gen
- Add more diagrams (Class, Sequence Diagrams)



References

- Clean Architecture
- Microservices Patterns by Chris Richardson
- Domain-Driven Design: Tackling Complexity in the Heart of Software by Eric Evans
- Implementing Domain-Driven Design by Vaughn Vernon
- https://vaadin.com/learn/tutorials/ddd/ddd_and_hexagonal



Thank you!

We hope to cooperate with your company.